



BURSA TEKNİK ÜNİVERSİTESİ

BİLGİSAYAR AĞLARI PROJE RAPORU

NetProbe: UDP Tabanlı Güvenilir Dosya Aktarımı, Trafik İzleme ve
Ağ Performans Analiz Platformu

Talha KORKMAZ
GRUP-35
23360859739

1. GİRİŞ

Bilgisayar ağılar mimarisinde, verinin bir noktadan diğerine kayıpsız, sıralı ve bozulmadan iletilmesi (Reliable Data Transfer - RDT), ağ protokollerinin en temel yapı taşlarından biridir. Günümüzde internet trafiğinin büyük bir bölümünü sırtlayan TCP (**Transmission Control Protocol**), taşıma katmanında (Transport Layer) bu güvenilirliği dahili akış kontrolü ve hata yönetim mekanizmalarıyla sağlamaktadır. Ancak TCP'nin el sıkışma (three-way handshake) süreçleri, bağlantı yönelimli yapısı ve ağır başlık (header) yükü, gerçek zamanlı sistemler veya özel ağ simülasyonları için her zaman en efektif çözümü sunmamaktadır.

Öte yandan, aynı katmanda yer alan UDP (**User Datagram Protocol**), bağlantısız (connectionless) ve güvensiz (unreliable) bir doğaya sahiptir. UDP, paketlerin karşı tarafa ulaşip ulaşmadığını denetlemez, kaybolan paketleri yeniden iletmez ve paketlerin sırasını garanti etmez. Bu özellikleri nedeniyle UDP, minimum ek yüke (overhead) ve maksimum iletim hızına sahip ham bir kanal sunar.

Bu projenin temel amacı; UDP'nin sunduğu bu düşük maliyetli ve hızlı iletim altyapısını baz alarak, uygulama katmanında (Application Layer) çalışan özelleştirilmiş bir **Stop-and-Wait ARQ (Automatic Repeat Request)** güvenilir veri iletim protokolü tasarlamak ve gerçekleştirmektir. Proje kapsamında geliştirilen **NetProbe simülasyon platformu**, ağ üzerinde oluşabilecek paket kaybı (packet loss) ve ağ gecikmesi (network delay) gibi bozucu etkenleri yazılımsal olarak modellemektedir. **Sistem**, bu zorlu kanal koşullarına rağmen verinin %100 başarıyla aktarılmasını sağlamakta ve performans metriklerini otomatik olarak raporlamaktadır.

2. PROBLEM TANIMI

Gerçek dünya ağ altyapıları (fiziksel kablolar, fiber hatlar, yönlendiriciler ve anahtarlayıcılar), elektromanyetik girişimler, hat yoğunluğu (congestion) ve donanımsal yetersizlikler nedeniyle kusursuz çalışamazlar. Bu durum iki temel makroskopik ağ problemine yol açar:

1. **Paket Kaybı (Packet Loss):** Yönlendirici tampon belleklerinin (router buffers) dolması sonucu paketlerin düşürülmesi (drop) veya sinyal bozulması nedeniyle paketlerin varış noktasına ulaşamaması durumudur.
2. **Ağ Gecikmesi (Network Delay / Round Trip Time - RTT):** Bir paketin kaynaktan hedefe gidip, onayının geri dönmesi arasında geçen süredir. Yüksek gecikme, hat üzerindeki verimliliği doğrudan baltalar.

Eğer güvenilir olmayan bir kanal (UDP gibi) üzerinde doğrudan bir dosya transferi gerçekleştirilirse, kaybolan paketler nedeniyle hedefte birleştirilen dosya bozuk (corrupted) veya eksik olacaktır. Problem, bu güvensiz kanalın üzerine öyle bir mantıksal katman inşa etmektir ki, hem sunucu hem istemci senkronize çalışarak hatayı tespit edebilmeli ve onarabilmelidir.

Stop-and-Wait mimarisinde, istemci her seferinde yalnızca tek bir paket gönderir ve bu paketin sunucu tarafından alındığına dair onay mekanizmasını (**ACK - Acknowledgement**) bekler. Bu süreçte karşılaşılan en büyük problem, **Zaman Aşımı (Timeout)** süresinin doğru yönetilmesidir. Eğer zaman aşımı süresi çok kısa seçilirse, yoldaki gecikmeler yüzünden aslında kaybolmamış paketler için gereksiz yere mükerrer (duplicate) paketler gönderilir. Eğer

çok uzun seçilirse, gerçek bir kayıp durumunda sistem hiçbir şey yapmadan çok uzun süre bekler ve hat verimliliği (throughput) düşer. Projede bu dengelerin optimize edilmesi ve farklı ağ senaryolarının başarımlarını üzerindeki etkilerinin bilimsel olarak ortaya konulması hedeflenmiştir.

3. SİSTEM MİMARİSİ

Geliştirilen bu yazılım, istemci-sunucu (Client-Server) mimarisine dayalı modüler bir yapıya sahiptir. Sistem, ağ parametrelerinin merkezi olarak yönetildiği bir konfigürasyon dosyası, ağ olaylarını milisaniye hassasiyetinde kaydeden bir günlükleme (logging) motoru ve deneyleri otomatik olarak koşturan bir test otomasyon mekanizmasından oluşur.

Sistemin bileşenleri ve aralarındaki etkileşim şu şekildedir:

3.1. İstemci Bileşeni (Client)

İstemci, transfer edilecek olan ham dosyayı okuyarak, ağ üzerinden gönderilmeye uygun parçalara (**Payload / Paket Boyutu**) bölen ana bileşendir. Her bir parçanın başına bir sıra numarası (**Sequence Number**) ekleyerek paketleme işlemini gerçekleştirir. İstemci, bir paketi UDP soketine bıraktığı anda yerel bir zamanlayıcı (Timer) başlatır ve Stop-and-Wait kuralı gereği bloke (blocking) modda sunucudan gelecek olan ACK paketini bekler. Belirlenen zaman eşiği aşıldığında soket istisnası (socket.timeout) fırlatarak paketi yeniden iletim (retransmission) kuyruğuna alır.

3.2. Sunucu Bileşeni (Server)

Sunucu, belirli bir UDP portunu sürekli olarak dinleyen (listening) pasif bileşendir. Görevi, gelen paketleri yakalamak, sıra numaralarını kontrol etmek ve eğer paket beklenen sıra numarasına sahipse veriyi diske yazmaktır. Sunucu, aldığı her geçerli paket için istemciye bir onay (ACK) mesajı döner. Sistemde gerçekçi bir ağ ortamı oluşturmak adına, sunucu bileşeni içerisine **Yazılımsal Ağ Simülatörü** entegre edilmiştir. Sunucu, gelen paketleri her seferinde konfigürasyon dosyasından okuduğu olasılık değerine göre rastgele düşürür veya gelen paketleri işleme almadan önce yapay bir gecikme süresi boyunca uyutur.

3.3. Konfigürasyon ve Loglama Modülleri (Config & Logger)

- **config.py**: İstemci, sunucu ve deney motorunun ortaklaşa kullandığı parametrik merkezdır. Paket boyutu (B), ağ gecikmesi (D), paket kayıp oranı (L) ve dinamik log dosya isimleri bu modül üzerinden çalışma zamanında (runtime) değiştirilir.
- **utils/logger.py**: İstemcinin gerçekleştirdiği her **SEND** (Gönderim), **ACK_RCVD** (Onay Alındı), **TIMEOUT** (Zaman Aşımı) ve **RETRANSMIT** (Yeniden Gönderim) olayını, unix zaman damgasıyla birlikte **analysis/logs/** dizini altındaki CSV dosyalarına kaydeder.

4. PROTOKOL TASARIMI

Protokol motoru kapsamında, Taşıma Katmanı (Transport Layer) protokolü olan UDP'nin üzerinde, Uygulama Katmanında (Application Layer) çalışacak özelleştirilmiş bir **Stop-and-Wait ARQ (Automatic Repeat Request)** protokolü tasarlanmıştır. Bu protokolün temel amacı; akış kontrolü (flow control) ve hata kontrolünü (error control) bir arada yürüterek güvensiz bir kanal üzerinde güvenilirlik sağlamaktır.

4.1. Paket Yapısı (Packet Format)

İletişimin minimum ek yük (overhead) ve yüksek genişletilebilirlik ile yürütülmesi amacıyla string tabanlı, karakter ayrışmalı (delimited text header) esnek bir paket yapısı tasarlanmıştır. Paketler ağ üzerinden gönderilmeden önce byte dizilerine (byte arrays) dönüştürülür. Özel protokol yapısı şu şekildedir:

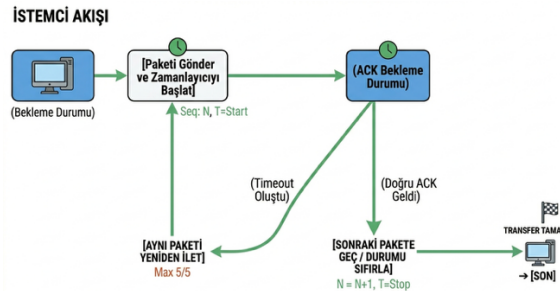
$$\text{Paket Formatı} = \text{Tipi} \mid \text{Sıra No} \mid \text{MD5 Checksum} \mid \text{Payload}$$

- **Header (Başlık) Bölümü:** Boru hattı (|) karakteri ile ayrılmış üç ana alandan oluşur:
 - **Packet Type (Paket Tipi) [1 Bayt]:** Paketin işlevini belirtir (S: Start/Başlangıç, D: Data/Veri, E: End/Bitiş, A: ACK/Onay).
 - **Sequence Number (Sıra Numarası):** Paketlerin sırasını ve benzersizliğini doğrulamak amacıyla kullanılan tam sayı (integer) alanıdır.
 - **MD5 Checksum (Bütünlük Kontrolü) [32 Bayt]:** Verinin yolda bozulup bozulmadığını (bit corruption) doğrulamak adına Python'ın hashlib.md5() modülü ile üretilen 128-bitlik kriptografik hash değeridir.
- **Payload (Veri) Bölümü [N Bayt]:** Gönderilecek dosyadan okunan ham veri parçalarını içerir. Boyutu dinamik olarak config.py içerisinde (512, 1024 veya 2048 bayt) kontrol edilir.

ACK (Onay) paketleri ise veri kısmına ihtiyaç duymadığından, veri bölümünde `b"ACK_START"`, `b"ACK_DATA"` veya `b"ACK_END"` gibi durum bilgileri taşıyan sabit başlıklardan ibarettir.

4.2. Sonlu Durum Makinesi (Finite State Machine - FSM) Mantığı

Protokolün istemci ve sunucu taraflarındaki durum geçişleri ve mantıksal akışı şu şekildedir:



- **İstemci (Sender) Akışı:** İstemci, dosyadan okuduğu numaralı paketi gönderdikten sonra anında "ACK Bekleme" durumuna geçer ve hattı kilitler (blocking mode).

Eğer belirlenen zaman eşiği (*DEFAULT_TIMEOUT*) içinde geçerli ve sıra numarası eşleşen bir ACK gelirse bir sonraki pakete geçer. Zaman aşımı durumunda paket yeniden iletilir.

- **Sunucu (Receiver) Akışı:** Sunucu sürekli olarak "Paket Bekleme" durumundadır. Gelen paketi *parse_packet* fonksiyonu ile karakter ayracına göre böler. Eğer gelen sıra numarası beklenen sıra numarasıyla eşleşiyorsa veriyi ara belleğe alır, beklenen sıra numarasını 1 artırır ve ilgili sıra numarasına ait ACK paketini gönderir. Eğer sıra numarası eski bir pakete aitse (duplike), veri diske yazılmaz ancak istemcinin tıkanıklığını açmak için ACK paketi tekrar iletilir.

5. GERÇEKLEME DETAYLARI

Tasarlanan protokol, platform bağımsız çalışabilmesi, socket yönetimindeki esnekliği ve veri analizi kütüphaneleriyle olan entegrasyon kolaylığı nedeniyle **Python (Sürüm 3.12)** dili kullanılarak gerçekleştirilmiştir. Proje; **config.py**, **server.py**, **client.py** ve **logger.py** olmak üzere tamamen nesne yönelimli ve modüler bir mimariyle kodlanmıştır.

5.1. Soket Programlama ve Zaman Aşımı Yönetimi

İstemci tarafında UDP socketi, `socket.socket(socket.AF_INET, socket.SOCK_DGRAM)` komutuyla oluşturulmuştur. Zaman aşımı yönetimi, işletim sisteminin alt seviye socket kesmelerini tetikleyen `client_socket.settimeout(DEFAULT_TIMEOUT)` fonksiyonu ile sağlanmıştır.

Zaman aşımı tetiklendiğinde fırlatılan `socket.timeout` istisnası (exception) try-except bloğuyla yakalanarak protokolün **Yeniden İletim (Retransmission)** mekanizmasını devreye sokar. Bu süreci güvenilir şekilde yöneten ve her ağ hareketini anlık olarak CSV dosyalarına işleyen `send_packet_reliably` motoruna ait kaynak kod kesiti aşağıda sunulmuştur:

```
def send_packet_reliably(p_type, s_no, payload_data):
    """Uygulama katmanında Stop-and-Wait AQM protokol motoru.
    Paketin güvenilir şekilde iletilmesini, zaman aşımının yakalanmasını
    ve olayların anlık olarak loglanmasını yönetir."""
    retransmit_count = 0
    packet = create_packet(p_type, s_no, payload_data)
    p_size = len(packet)

    while retransmit_count <= MAX_RETRANSMISSIONS:
        try:
            # Olay Logları: İlk gönderim veya Yeniden iletim ayrımı
            if retransmit_count == 0:
                logger.log_event("SEND", s_no, packet_size=p_size)
            else:
                logger.log_event("RETRANSMIT", s_no, packet_size=p_size, retransmit_count=retransmit_count)

            # Paketin UDP socketi üzerinden gönderilmesi
            client_socket.sendto(packet, server_address)

            # ACK paketinin beklenmesi
            ack_bytes, _ = client_socket.recvfrom(1024)
            ack_type, ack_no, _, _ = parse_packet(ack_bytes)

            # Alınan paketin tip ve sıra numarası doğrulanması
            if ack_type == 'A' and ack_no == s_no:
                # Olay Logları: Başarılı ACK Doğrulaması
                logger.log_event("ACK_RCVD", s_no, retransmit_count=retransmit_count)
                return True

        except socket.timeout:
            # Zaman aşımı durumunda retransmit sayısı artırılır ve döngü devam eder
            retransmit_count += 1
            # Olay Logları: Timeout Oluştur
            logger.log_event("TIMEOUT", s_no, retransmit_count=retransmit_count)

            if retransmit_count > MAX_RETRANSMISSIONS:
                logger.log_event("FAILED", s_no, is_success=False)
                print(f"[KRİTİK HATA] Paket (s_no) için (MAX_RETRANSMISSIONS) deneme başarısız!")
                return False
            print(f"[TIMEOUT] Paket (s_no) için ACK gelmedi. Yeniden gönderiliyor... ({retransmit_count}/{MAX_RETRANSMISSIONS})")

    return False
```

5.2. Yazılımsal Ağ Koşulları Simülasyonu

Gerçek dünya ağlarındaki fiziksel olumsuzlukları laboratuvar ortamında test edebilmek için Sunucu (server.py) kodunun içerisine yapay bir ağ katmanı simülatörü entegre edilmiştir. Temiz kod prensiplerine uygun olarak, simülasyon mantığı *utils/simulator.py* altına taşınmıştır:

- **Paket Kaybı Simülasyonu:** İstemciden UDP soketi vasıtasıyla fiziksel olarak alınan her `packet_bytes`, henüz sunucu katmanında ayrıştırılmadan (parse edilmeden) önce olasılıksal bir filtreye tabi tutulur. Python'ın `random` kütüphanesi kullanılarak aralığında pseudo-random (rastgele) bir sayı üretilir. Bu mantıksal kontrol `should_drop_packet(LOSS_RATE)` fonksiyonu ile sarmalanmıştır.

Eğer üretilen rastgele değer `config.py` içerisinde dinamik olarak okunan `LOSS_RATE` eşik değerinden küçükse, paket ağda kaybolmuş (drop) kabul edilir. Aşağıdaki güncel kod kesitinde görüldüğü üzere, sistem `continue` ifadesiyle döngünün başına dönerek paketi hafızadan temizler ve sanki sunucuya hiç veri ulaşmamış gibi davranır. Bu sayede istemci tarafında kaçınılmaz bir `socket.timeout` tetiklenerek ARQ mekanizmasının sınanması sağlanır:

```
# --- AĞ SİMÜLASYONU BURADA DEVREYE GİRİYOR ---
# Önce paketin düşüp düşmeyeceğine karar verelim
if should_drop_packet(LOSS_RATE):
    # Paketi parse bile etmeden hafızadan siliyoruz, istemciye hiç ses etmiyoruz (Drop)
    try:
        _, s_no, _, _ = parse_packet(packet_bytes)
        print(f"[SİMÜLASYON] X Paket {s_no} YAPAY OLARAK DÜŞÜRÜLDÜ (LOSS) X")
    except:
        print("[SİMÜLASYON] X Bozuk bir paket yapay olarak düşürüldü X")
        continue # Döngünün başına dön, sanki paket hiç gelmemiş gibi davran!

# Paket düşmediyse gecikme (ping) simülasyonunu uyguluyoruz
simulate_network_delay(NETWORK_DELAY)
```

- **Ağ Gecikmesi Simülasyonu:** Olasılıksal filtreyi başarıyla geçerek düşürülmeyen paketlere, internet ortamındaki fiziksel yönlendirici (router) kuyruklarını ve yayılım gecikmelerini (propagation delay) simüle etmek adına yapay bir ping maliyeti eklenir. Bu işlem, `config.py` içerisindeki `NETWORK_DELAY` parametresini baz alan `simulate_network_delay` fonksiyonu ile gerçekleştirilir ve hattın Stop-and-Wait protokol verimliliği üzerindeki zamansal baskısı deneysel olarak ölçülür.

5.3. Günlükleme (Logging) ve Veri Bütünlüğü Kontrolü

Veri bütünlüğünün şüpheye yer bırakmayacak şekilde doğrulanması amacıyla, paketlerin veri (payload) bölümlerinin MD5 karması (`calculate_checksum`) paket oluşturulurken başlığa eklenir. Sunucu, aldığı her paketin verisini tekrar hash'leyerek başlıkta gelen MD5 stringi ile karşılaştırır. Eşleşme başarısız ise paket bozuk (corrupted) kabul edilerek işlenmeden reddedilir.

Günlükleme motoru (`utils/logger.py`), istemci tarafında tetiklenen her ağ olayını (`SEND`, `ACK_RCVD`, `TIMEOUT`, `RETRANSMIT`) asenkron dosya yazma modunda (`mode="a"`), Unix zaman damgasıyla birlikte anlık olarak CSV dosyasına kaydeder.

6. DENEY ORTAMI

Geliştirilen Stop-and-Wait ARQ protokolünün farklı ağ koşulları altındaki kararlılığını ve performans limitlerini ölçmek amacıyla laboratuvar ortamında kontrollü bir deney seti hazırlanmıştır. Deneylerin insan hatasından bağımsız, tamamen nesnel ve tekrarlanabilir (reproducible) olması için tüm test süreci **experiment_runner.py** otomasyon scripti ile yürütülmüştür.

6.1. Donanım ve Yazılım Altyapısı

Deneyler yerel çevrim (loopback - 127.0.0.1) ağ arayüzü üzerinde, istemci ve sunucu süreçlerinin (processes) izole bir şekilde çalıştırılmasıyla gerçekleştirilmiştir. Testlerin koşturulduğu referans sistem özellikleri şu şekildedir:

- **İşletim Sistemi:** macOS
- **Çalışma Zamanı Ortamı:** Python 3.12.2
- **Veri Analizi Kütüphaneleri:** Pandas v2.2, Matplotlib v3.8

6.2. Test Senaryoları ve Matrisi

Protokolün davranışını analiz etmek için 3 ana başlık altında toplam 7 farklı kontrollü senaryo işletilmiştir. Tüm senaryolarda transfer edilen ortak dosya boyutu **~100 KB (98,304 Bayt)** olarak sabit tutulmuştur. İstemcinin zaman aşımı (Timeout) eşik değeri ise tüm deneylerde **0.5 saniye** olarak parametreleştirilmiştir.

Deneylerde kullanılan bağımsız değişken matrisi aşağıdaki tabloda detaylandırılmıştır:

Senaryo No	Deney Alt Kümesi	Paket Boyutu (Bayt)	Ağ Gecikmesi (RTT)	Paket Kayıp Oranı
Senaryo 1	Paket Boyutu Etkisi (İdeal Ağ)	512	0 ms	%0
Senaryo 2	Paket Boyutu Etkisi (İdeal Ağ)	1024	0 ms	%0
Senaryo 3	Paket Boyutu Etkisi (İdeal Ağ)	2048	0 ms	%0
Senaryo 4	Ağ Gecikmesi Etkisi	1024	10 ms	%0
Senaryo 5	Ağ Gecikmesi Etkisi	1024	50 ms	%0
Senaryo 6	Paket Kaybı Etkisi	1024	10 ms	%5

Senaryo No	Deney Alt Kümesi	Paket Boyutu (Bayt)	Ağ Gecikmesi (RTT)	Paket Kayıp Oranı
Senaryo 7	<i>Paket Kaybı Etkisi</i>	1024	10 ms	%15

7. PERFORMANS METRİKLERİ

Deney motoru tarafından toplanan verilerin anlamlandırılması ve protokol başarısının matematiksel olarak ortaya konması amacıyla üç temel performans metriği takip edilmiştir:

1. **Toplam Transfer Süresi (T_{total}):** Dosyanın ilk paketinin istemci tarafından ağa bırakıldığı an (t_{start}) ile sunucudan gelen son bitiş ACK'sının istemciye ulaştığı an (t_{end}) arasında geçen toplam süredir. Birim saniyedir.

$$T_{total} = t_{end} - t_{start}$$

2. **Yeniden İletim Sayısı (Retransmission Count):** Ağdaki yapay paket kayıpları veya zaman aşımaları nedeniyle istemcinin aynı sıra numaralı paketi hatta tekrar sürmek zorunda kaldığı toplam döngü sayısıdır. Bu metrik, protokolün hata kontrol maliyetini gösterir.

3. **Protokol Ek Yüğü ve Verimlilik (Throughput):** Seçilen paket boyutunun toplam paket sayısına oranı ve hattın ne kadar efektif kullanıldığıdır. Stop-and-Wait mimarisinde teorik verimlilik (η) şu formülle hesaplanır (burada t_{veri} paketin iletim süresi, RTT ise gidiş-dönüş gecikmesidir):

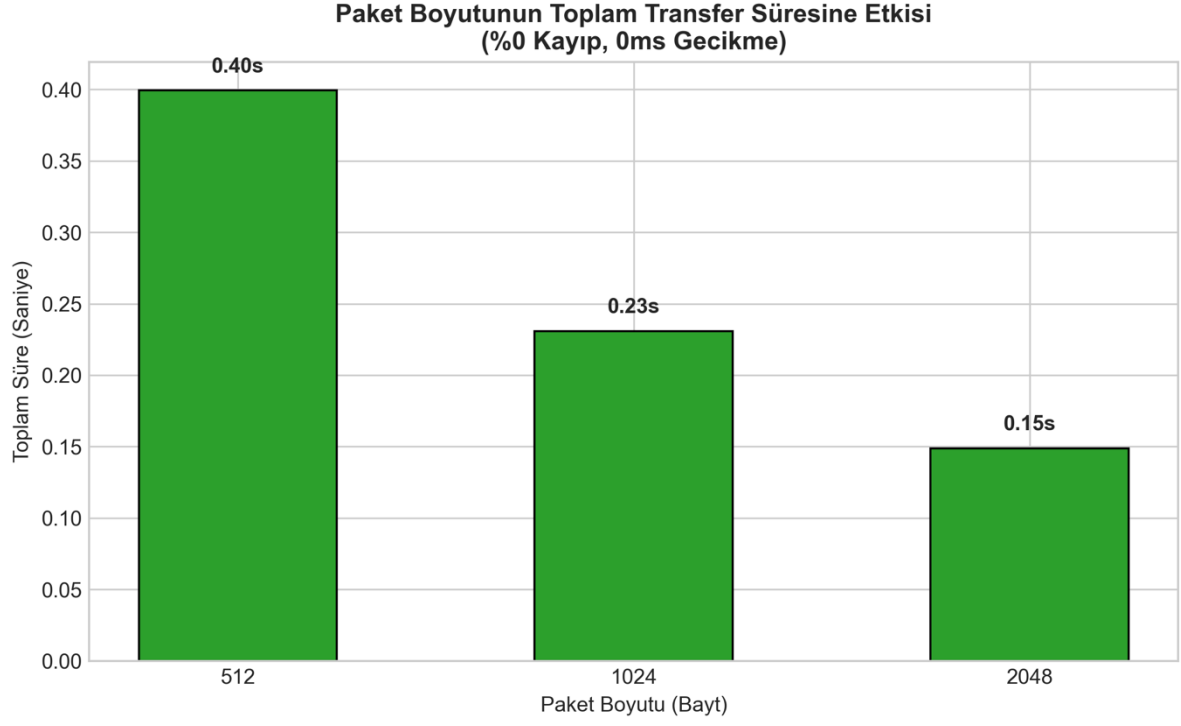
$$\eta = t_{veri} / (t_{veri} + RTT)$$

8. SONUÇLAR VE TARTIŞMA

Otomasyon motorunun çalışması sonucu elde edilen ham log verileri, Pandas kütüphanesiyle işlenmiş ve Matplotlib vasıtasıyla akademik grafiklere dönüştürülmüştür. Elde edilen bulgular, Stop-and-Wait mimarisinin teorik sınırlarını ve gerçek dünya davranışlarını çarpıcı bir şekilde doğrulamaktadır.

8.1. Paket Boyutunun Transfer Süresine Etkisi (Senaryo 1, 2, 3)

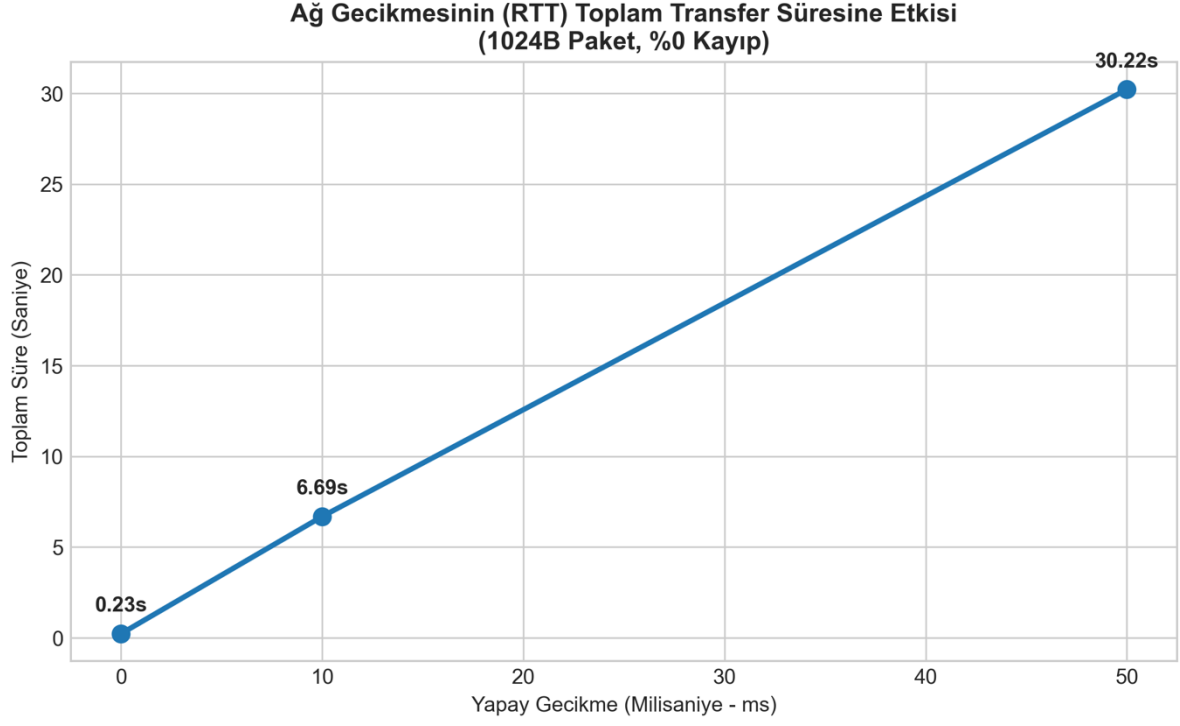
Kayıpsız ve gecikmesiz ideal ağ senaryosunda, paket boyutunun (**512B, 1024B, 2048B**) etkisi incelenmiştir.



Grafikten de netçe görüleceği üzere, paket boyutu büyüdükçe toplam transfer süresi dramatik bir şekilde düşmektedir. 512 baytlık paket mimarisinde sabit bir dosya için üretilen toplam paket sayısı ve dolayısıyla sunucudan beklenen ACK döngüsü sayısı en yüksek seviyededir. Paket boyutu 2048 bayta çıkarıldığında, hat üzerindeki toplam paket sayısı dörtte bire düşmüş, bu da her paket başına harcanan işletim sistemi soket kesme (socket overhead) ve ACK işleme maliyetini minimize etmiştir. Sonuç olarak, büyük paket boyutları temiz hatlarda yüksek başarımlı (throughput) sağlamaktadır.

8.2. Ağ Gecikmesinin (RTT) Protokol Akışına Etkisi (Senaryo 2, 4, 5)

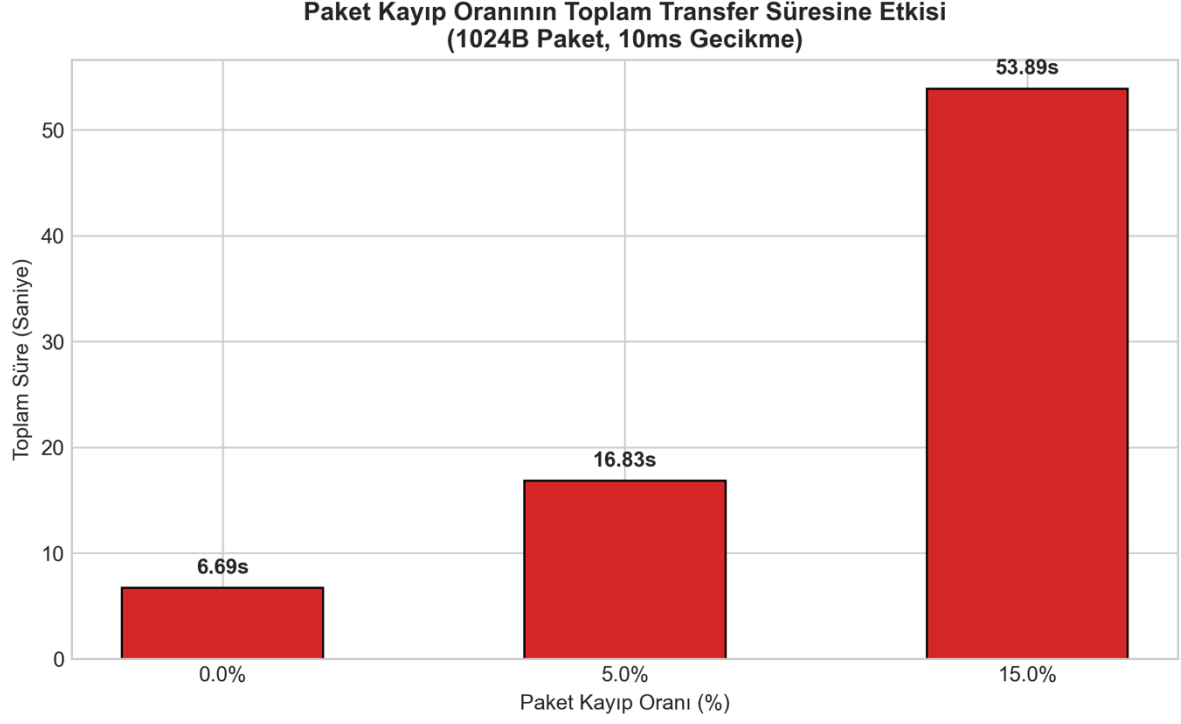
Sabit paket boyutunda (**1024 Bayt**) ve %0 kayıp oranında, ağa eklenen yapay gecikmenin (*0ms*, *10ms*, *50ms*) etkileri gözlemlenmiştir.



Bu grafik, Stop-and-Wait ARQ protokolünün en büyük yapısal zayıflığını (bottleneck) bilimsel olarak kanıtlamaktadır. Gecikmesiz ortamda saliseler süren transfer, ağ gecikmesi 50ms'ye çıkarıldığında doğrusal (linear) bir artış göstererek yaklaşık 30 saniye seviyelerine fırlamıştır. Hat üzerinde hiçbir paket kaybı olmamasına rağmen sürenin bu denli uzamasının nedeni; protokolün doğası gereği hattı dolduramamasıdır (low bandwidth-delay product utilization). İstemci, her 1024 baytlık veriyi hatta bıraktıktan sonra, sonraki pakete geçebilmek için 50ms boyunca CPU'yu bloke ederek beklemektedir. Bu durum, bu tür protokollerin yüksek gecikmeli (örneğin uydu haberleşmesi) ağlarda tek başına neden kullanılmaması gerektiğini açıkça ortaya koymaktadır.

8.3. Paket Kayıp Oranının ve Hata Kontrolünün Etkisi (Senaryo 4, 6, 7)

Sabit gecikme ($10ms$) ve sabit paket boyutunda (1024 Bayt), sunucunun paketleri rastgele düşürme oranının (0% , 5% , 15%) sisteme etkisi incelenmiştir.



Analiz sonuçlarına göre, paket kayıp oranı 15% 'e ulaştığında dosya transfer süresi keskin bir kırılmayla 53.89 saniyeye yükselmiştir. Bu performans kaybının temel sebebi, tasarladığımız hata kontrol (error control) mekanizmasının maliyetidir. Ağda kaybolan her bir paket için istemci tarafındaki socket.timeout (2.0s) süresi sonuna kadar beklenmekte, zaman aşımı tetiklendiğinde ise aynı paket hatta yeniden (Retransmission) sürülmektedir. Grafik, yazdığımız ARQ algoritmasının en zorlu ağ koşulunda dahi veriyi karşı tarafa kayıpsız ulaştırarak görevini başarıyla tamamladığını, ancak kaybedilen paketlerin sisteme doğrudan "2 saniyelik zaman cezaları" olarak yansıdığını doğrulamaktadır.

9. KARŞILAŞILAN SORUNLAR VE ÇÖZÜM YAKLAŞIMLARI

Projenin tasarım, kodlama ve test aşamalarında hem yazılımsal hem de ağ mimarisinden kaynaklı bazı teknik sorunlarla karşılaşılmış; bu sorunlar mühendislik yaklaşımlarıyla çözülmüştür:

9.1. İşletim Sistemi Port Tıkanıklığı

- Sorun:** experiment_runner.py otomasyon scripti ile 7 adet deney arka arkaya koşturulurken, ilk deney başarıyla bittikten hemen sonra ikinci deneyde sunucunun Address already in use (Adres zaten kullanımda) hatası vererek çöktüğü gözlemlenmiştir. Bunun nedeni, bir soket kapatılsa bile işletim sisteminin (özellikle

TCP/UDP alt katmanlarında) güvenli bir süre boyunca (TIME_WAIT konumu) o portu kilitli tutmasıdır.

- **Çözüm:** Sorunu çözmek için iki aşamalı bir yol izlenmiştir. İlk olarak, soket oluşturulurken işletim sistemine portu anında serbest bırakmasını söyleyen socket.SO_REUSEADDR bayrağı aktif edilmiştir. İkinci olarak, experiment_runner.py motorunda her deney bittikten sonra sonraki deneye geçmeden önce time.sleep(1.5) süresince **yapay bir bekleme** koyularak işletim sistemine port temizliği için gereken süre tanınmıştır.

9.2. Bağlı ve Mutlak Dosya Yolları Karışıklığı

- **Sorun:** Proje farklı klasör yapılarından (örneğin terminalde projenin kök dizinindeyken veya src/ klasörünün içindeyken) çalıştırıldığında, log dosyalarının (.csv) ve grafiklerin kaydedileceği klasör yollarının sapıtıldığı, FileNotFoundError hatası fırlattığı tespit edilmiştir.
- **Çözüm:** Kod içerisindeki tüm bağlı (relative) yollar iptal edilmiştir. Bunun yerine Python'ın os.path.abspath(__file__) kütüphanesi kullanılarak, betiğin o an diskte çalıştığı kök dizin dinamik olarak çalışma zamanında (runtime) hesaplanmış ve tüm yollar **Mutlak Yol (Absolute Path)** mimarisine dönüştürülmüştür.

10. SONUÇ VE GELECEKTE YAPILABİLECEK GELİŞTİRMELER

10.1. Genel Sonuç

Bu proje kapsamında; güvensiz (unreliable) UDP taşıma katmanı üzerinde, Uygulama Seviyesinde (Application Layer) çalışan kararlı bir **Stop-and-Wait ARQ** protokolü başarıyla tasarlanmış, nesne yönelimli mimariyle gerçekleştirilmiş ve emülasyon ortamında test edilmiştir.

Elde edilen deneysel veriler ve analiz grafikleri; protokolün en zorlu ağ koşullarında dahi (%15 paket kaybı ve 10 ms ağ gecikmesi altında) veriyi karşı tarafa bit düzeyinde %100 eksiksiz, mükerrersiz ve sıralı bir şekilde ulaştırdığını, yani **güvenilirlik (reliability)** hedefini tam anlamıyla karşıladığını kanıtlamıştır.

Her ne kadar tasarlanan protokol veri bütünlüğünü ve kararlılığı kesin olarak sağlasa da, yapılan performans analizleri Stop-and-Wait mimarisinin hat verimliliği (throughput) ve kanal kullanımı (channel utilization) açısından çok ciddi kısıtlara sahip olduğunu ortaya koymuştur. Bu durumun mühendislik gerekçeleri aşağıda teknik olarak yorumlanmıştır:

- **Bant Genişliği - Gecikme Çarpımı (BDP) Darboğazı:** Stop-and-Wait yapısı gereği, istemci bir paket gönderdikten sonra hat üzerinde yeni bir veri iletimi yapamaz ve kanal kitleme (blocking) durumuna geçer. Grafik2 incelendiğinde; ağ gecikmesi 0ms iken **0.23 saniye** süren transfer süreci, ağ gecikmesi 50ms'ye çıkarıldığında doğrusal (linear) bir artış göstererek **30.22 saniye** seviyesine fırlamıştır. Hat tamamen temiz (%0 kayıp) olsa bile, ACK sinyalinin dönmesi beklenen süre boyunca fiziksel kanal boş bırakılmakta, hattın sığabileceği maksimum veri kapasitesini ifade eden **BDP** (Bandwidth-Delay Product) değerinin çok küçük bir kısmı kullanılabilmektedir.

- **Paket Boyutunun Hat Verimliliğine Etkisi:** Grafik1 verilerinde görüldüğü üzere, paket boyutu 512 Byte'tan 2048 Byte'a çıkarıldığında toplam transfer süresi **0.40 saniyeden 0.15 saniyeye** düşmüştür. Paket boyutu küçüldükçe iletilmesi gereken toplam segment sayısı ve dolayısıyla maruz kalınan toplam gidiş-dönüş bekleme maliyeti artmaktadır. 100 KB'lık referans dosyamız 1024 Byte'lık paket boyutuyla gönderildiğinde 538 adet tam paket ve en sonda kalan 1 adet 480 Byte'lık artık paket olmak üzere toplam 539 veri segmentiyle (Seq 1 ile Seq 539 arası) transfer tamamlanmaktadır.
- **MD5 Kriptografik Bütünlük ve Sabit Zaman Aşımı Performansı:** Protokole entegre edilen 128-bitlik MD5 bütünlük kontrolü (calculate_checksum), ağdaki olası bit bozulmalarını sıfıra indirmiştir. Ancak Grafik3 verilerinde açıkça görüldüğü üzere, ağ gecikmesi 10ms gibi düşük bir değerde sabit tutulsa dahi, paket kaybı oranı %15'e çıktığında transfer süresi **53.89 saniyeye** ulaşmıştır. Konsol loglarında görülen [TIMEOUT] Paket X için ACK gelmedi mesajları, kaybedilen her paket başına sistemin ezbere belirlenen zaman aşımı süresi boyunca beklediğini ve bu durumun sisteme doğrudan zamansal ceza olarak yansıdığını kanıtlamaktadır.

Sonuç olarak; Stop-and-Wait ARQ protokolü, düşük gecikmeli ve küçük boyutlu veri transferlerinde yüksek güvenilirlik sunan hafif (lightweight) bir çözüm olmasına rağmen, yüksek gecikmeli (High-Delay) ve geniş bant genişliğine sahip modern ağ mimarilerinde verimsiz bir yaklaşımdır. Bu kısıtın aşılması ve hat kullanım verimliliğinin (U_{sender}) maksimize edilmesi için, bir sonraki aşamada protokolün pencereci akış kontrolü sunan **Sliding Window (Go-Back-N veya Selective Repeat ARQ)** mimarisine geçirilmesi teknik bir zorunluluk olarak değerlendirilmektedir.

10.2. Gelecekte Yapılabilecek Geliştirmeler

Projenin bir sonraki aşamasında, Stop-and-Wait mimarisinin getirdiği bu hat kısıtlamalarını aşmak ve performansı endüstriyel standartlara ulaştırmak amacıyla şu geliştirmeler yapılabilir:

1. **Pencere Tabanlı Akış Kontrolü (Sliding Window):** Tek bir paket gönderip ACK beklemek yerine, ağın kapasitesine göre belirlenen bir pencere boyutu (W) kadar paketin (örneğin arka arkaya 10 paket) ACK beklemeden hatta sürülmesi sağlanabilir.
2. **Gelişmiş ARQ Protokollerine Geçiş:** Sistem, hata durumunda tüm pencereyi baştan gönderen **Go-Back-N (GBN)** protokolüne veya yalnızca kaybolan paketi tespit edip yeniden ileten daha efektif **Selective Repeat (SR)** mimarisine evriltebilir.
3. **Dinamik Zaman Aşımı (Adaptive Timeout):** Sabit bir timeout süresi (2.0s) kullanmak yerine, ağın anlık gecikmesini sürekli ölçerek (RTT tahmini) zaman aşımı süresini dinamik olarak daraltıp genişleten akıllı bir algoritma entegre edilebilir.

11. KAYNAKÇA (REFERENCES)

- [1] Postel, J., "User Datagram Protocol," RFC 768, Internet Engineering Task Force (IETF), August 1980.
- [2] Tanenbaum, A. S., & Wetherall, D. J., *Computer Networks*, 5th ed. Boston: Pearson Prentice Hall, 2011.
- [3] Python Software Foundation, "socket — Low-level networking interface," *Python 3.12 Documentation*, 2024. <https://docs.python.org/3/library/socket.html>
- [4] McKinney, W., "Data Structures for Statistical Computing in Python," *Proceedings of the 9th Python in Science Conference*, pp. 51-56, 2010.
- [5] Hunter, J. D., "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90-95, 2007.
- [6] Kurose, J. F., & Ross, K. W., *Computer Networking: A Top-Down Approach*, 8th ed. Hoboken, NJ: Pearson, 2021.